

Hi, I'm

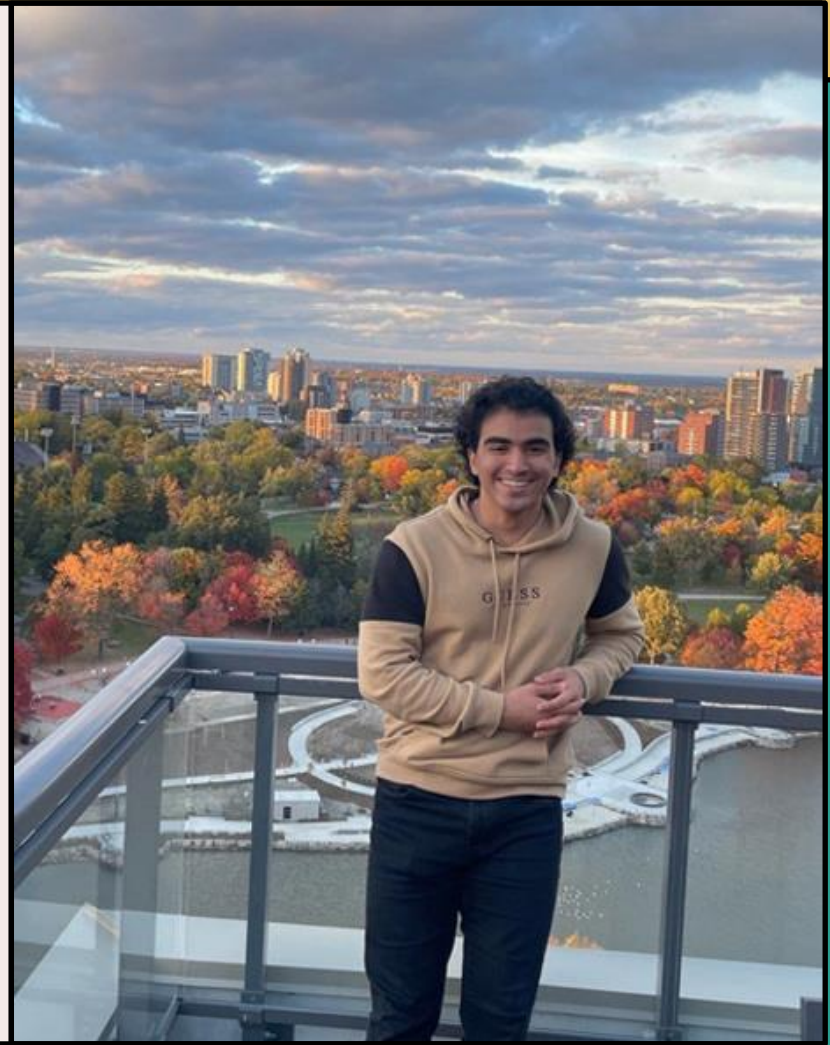
Hossein Molavi

New Grad Comp Eng student @ UWaterloo

Passionate about hardware, software and the intersection which they meet!

Seeking NewGrad positions to contribute to innovative projects.

Excited to bring my skills to your team!

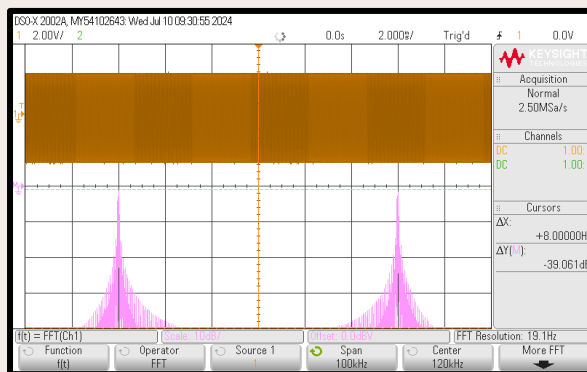


Bachelors of Computer Engineering Class of '26 with GPA 3.7

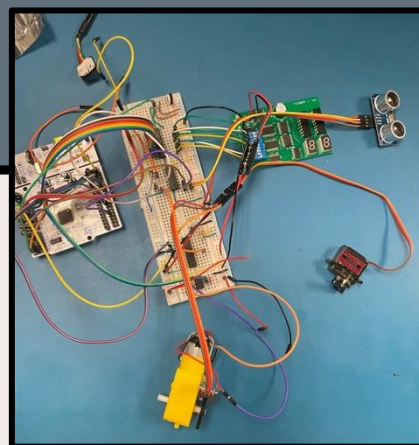
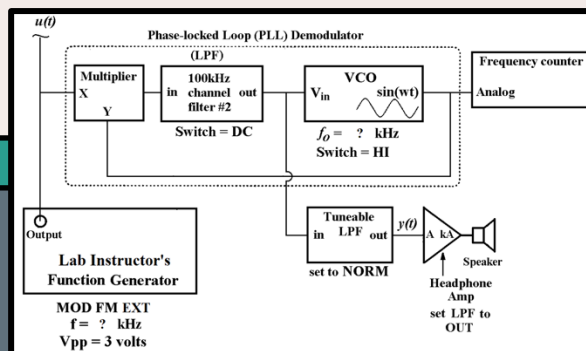
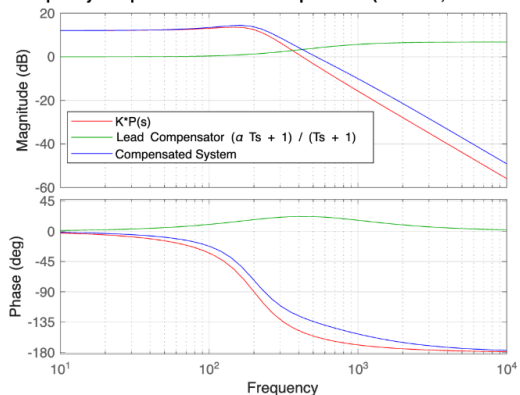
Sept. 2021 – Aug. 2026

Favorite courses:

- Real Time Operating Systems (C)
- Algorithms & Data Structures (C++)
- Systems & Concurrency (C)
- Digital Signals Processing
- Computer Security (Python)
- Machine Learning (Python)
- Analog Control Systems (MATLAB)



Frequency Response with Lead Compensator ($\alpha = 2.19$, $T = 0.001577$)

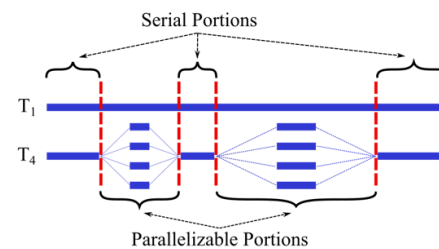


ECE 298 final project demo
The Water Reservoir Project



E7 building – My 2nd home during study terms

Amdahl's Law



Wireless Developer Contractor

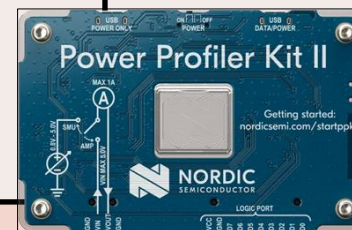
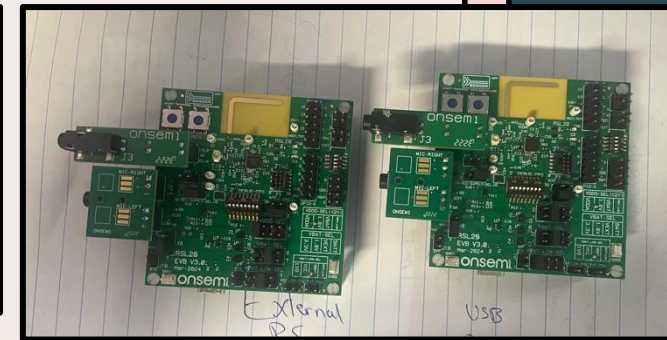
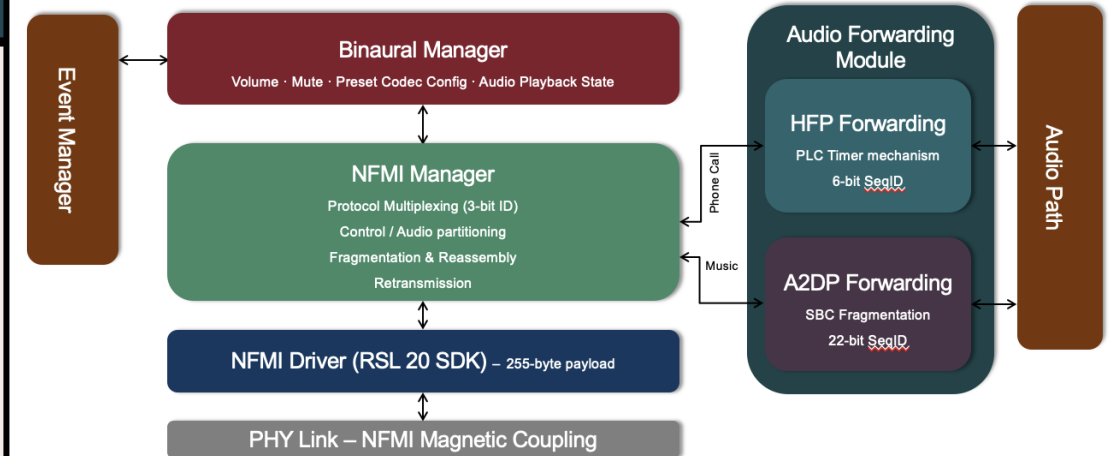
onsemi

Apr. 2026 – Present



- Architected an autonomous development ecosystem utilizing a multi-agent AI framework (Orchestrator, Architect, and 8 specialized agents) to streamline firmware development cycles
- Developed a core automation engine leveraging CMake and J-Link for concurrent multi-device firmware deployment, providing the AI agents with real-time, timestamped hardware telemetry
- Integrated AI agents with Bitbucket and Jira via MCP servers, enabling autonomous task analysis, hardware flashing, and diagnostic log validation in a steady-state testing environment

NFMI Architecture (OTC)



Wireless Developer Intern

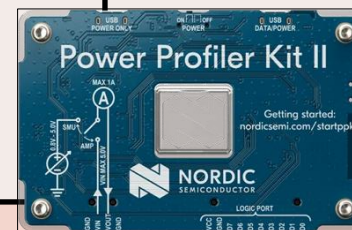
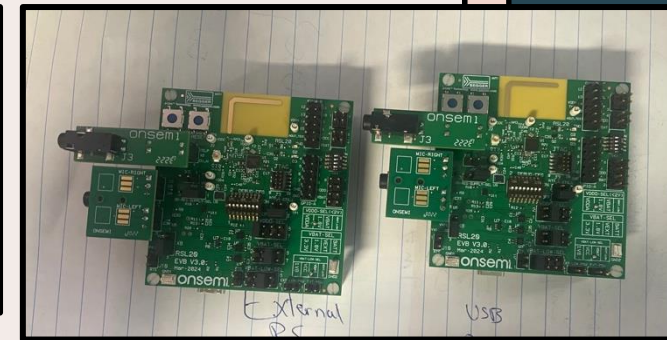
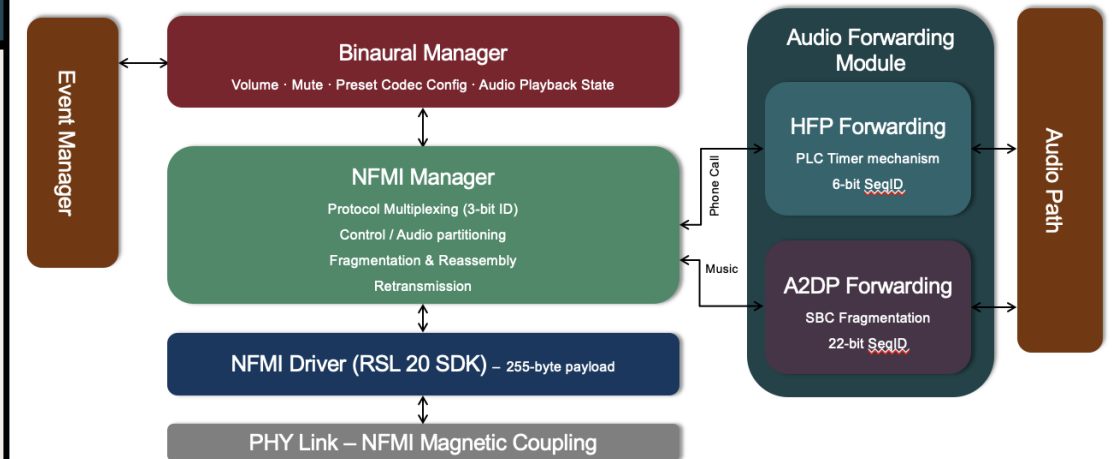
onsemi

Sept. 2025 – Apr. 2026



- Contributed to the release of RSL20 SDK & Hearing-Aid platform
- Hardened NFMI driver by resolving DMA race conditions causing RX buffer overruns and corrupt audio payloads
- Built wireless audio interfaces supporting HFP/A2DP profiles with mSBC and CVSD codec integration
- Developed NfmiManager for ear-to-ear audio forwarding with SBC frame fragmentation and PLC insertion
- Engineered BinauralManager for guaranteed payload delivery with ACK-based retransmission protocol
- Wrote 580+ unit tests using CppUTest/gcov achieving 100% code coverage across NFMI application modules
- Captured HCI logs and Ellisys sniffs to reverse-engineer BT Classic/LE Audio in commercial earbuds

NFMI Architecture (OTC)

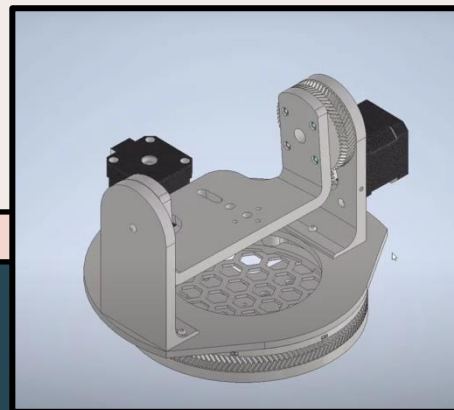
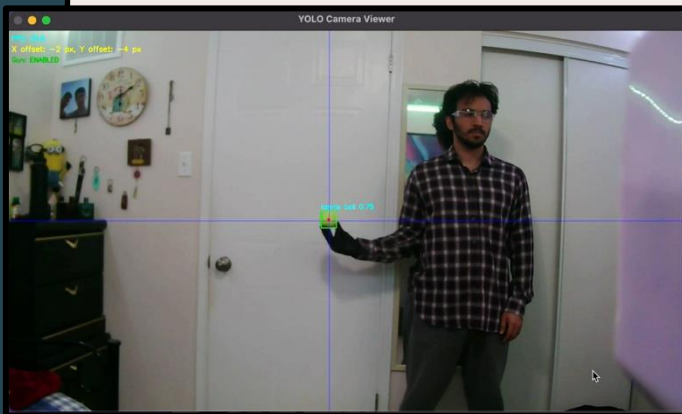
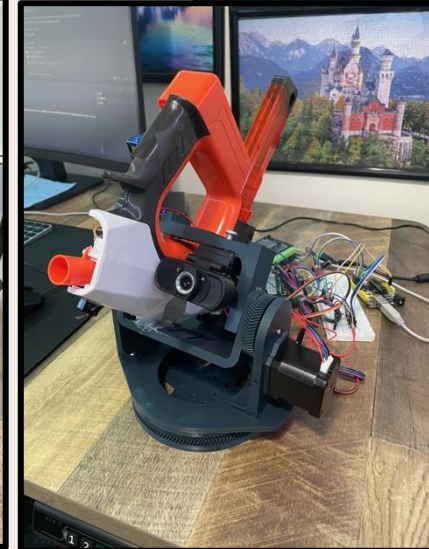
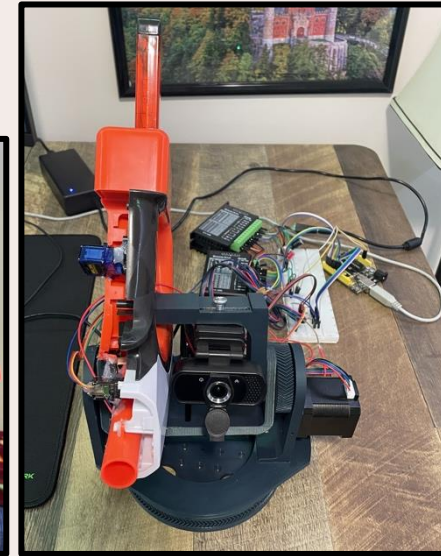


Co-founder & Firmware Engineer Asymptote Labs

Apr. 2025 – Present [Demo video link](#)

- Designed and built an embedded mechatronic turret integrating a webcam, custom YOLO detector, and Arduino-controlled NEMA17 steppers (TB6600 drivers) to hit objects via modified nerf-gun
- Implemented a robust real-time serial command protocol and a non-blocking Arduino firmware that executes absolute-position step moves, homing via limit switches
- Engineered PC to Arduino control loop sending commands at 5 Hz to balance responsiveness and serial reliability
- Achieved high positional precision using 1/16 micro stepping and gear reduction resulting in accuracy at 100 m of ~ 2.32 cm (yaw) and ~ 6.44 cm (pitch)

Asymptote Labs
Non-lethal Anti-Drone Defense Systems



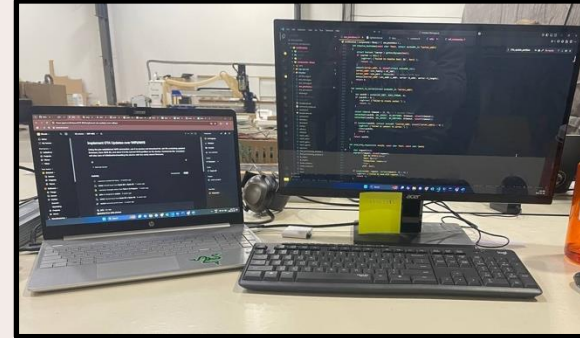
Firmware Engineering Intern

Wrmth Corp

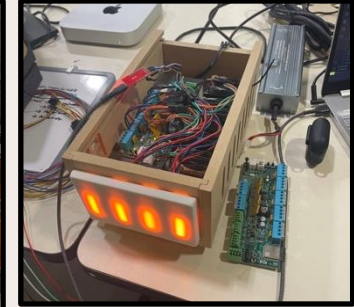
Jan. 2025 – Apr. 2025

- Implemented fail-safe AWS OTA system with signature validation & anti-rollback mechanism for the ESP32s3 based chipset
- Architected power-efficient wireless drivers for secure AWS IoT telemetry comms via MQTT sending real-time chair data
- Integrated AWS IoT Core fleet provisioning using PKI
- Resolved complex memory race conditions via JTAG debugging

The Team !



Working on OTA feature



Testing new features on developer boards



Lunch break vibes



PCB Used in Wrmth Chairs
(ESP32 based)

Embedded Systems Programmer

Midnight Sun

Aug. 2023 – Dec. 2023

MIDNIGHT  SUN

- Contributed to Midnight Sun's 15th generation solar car (MSXV), competed in the American Solar Car Challenge 2024 ([link](#))
- Architected reusable FreeRTOS wrappers, enhancing real-time task scheduling and management
- Implemented high-throughput sensor data acquisition using DMA-driven multi-channel ADC on STM32
- Boosted reliability via HIL simulation, thread-safe design & advanced hard-fault diagnostics



15th Generation Solar Car at Competition



Debugging Power Distribution



American Solar Car Challenge
Summer 2024, NCM Motorsports Park KY, USA

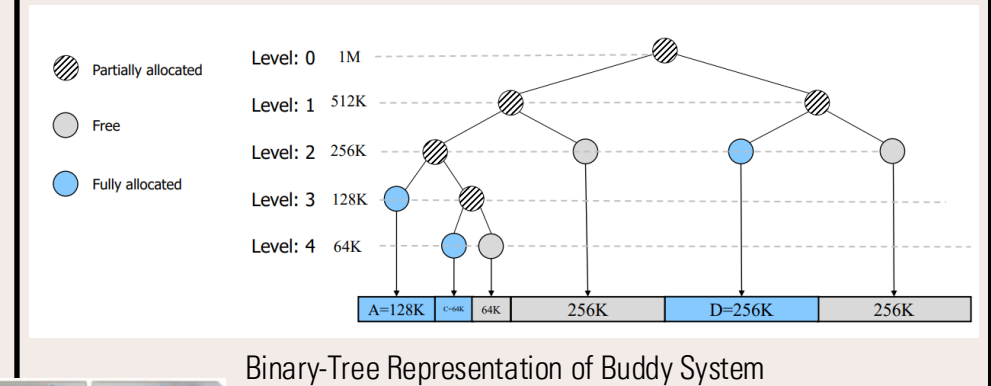
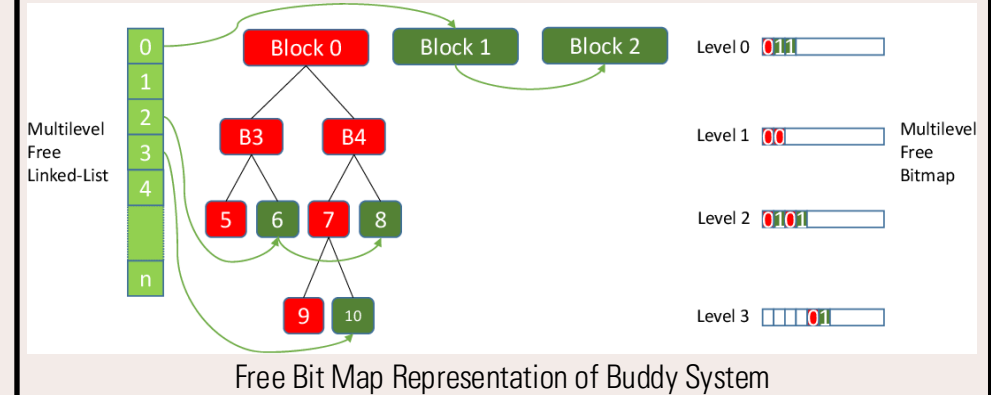
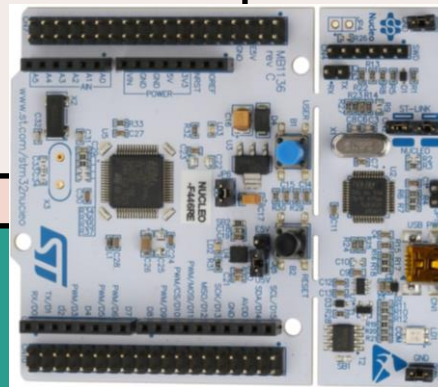
Race Documentary [here](#)



Real-Time OS

Code can be provided upon request

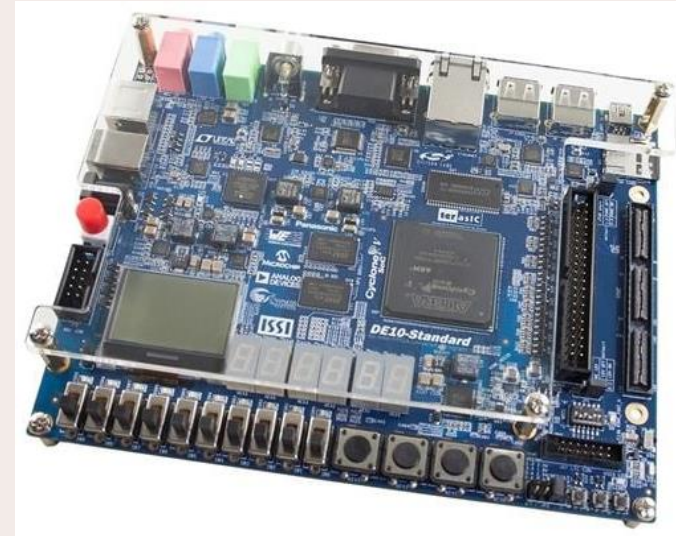
- Developed a pre-emptive real-time operating system which employs an EDF scheduling algorithm, supports up to 16 tasks, and has task-based user stack allocations and dynamic memory allocation
- Managing (alloc & dealloc) 32KB heap space using Buddy memory allocation and free lists
- Implementing SVC/PendSV-based context switching and ARM assembly handlers to enable preemptive task switching and kernel-mode entry
- Integrating allocator with task lifecycle by allocating stacks in task creation and reclaiming memory on task exit
- Using various techniques to optimize memory allocation runtime close to malloc in standard library
- Written in C and ARM



Stereo Audio System

Code can be provided upon request

- Architected an embedded stereo system on an Intel Cyclone V SoC FPGA, utilizing the Avalon bus to facilitate seamless data transfer between the soft-core processor and audio codec peripherals
- Implemented a multi-buffered queuing system to manage decoded MP3 data streams, ensuring continuous and glitch-free audio playback by preventing FIFO underrun
- Developed a custom audio driver in embedded C to interface with a hardware audio codec, programming independent left and right audio channels for stereo sound output
- Programmed a finite state machine (FSM) to manage a real-time data pipeline, orchestrating the queuing and transmission of audio samples to dedicated left and right speaker channels



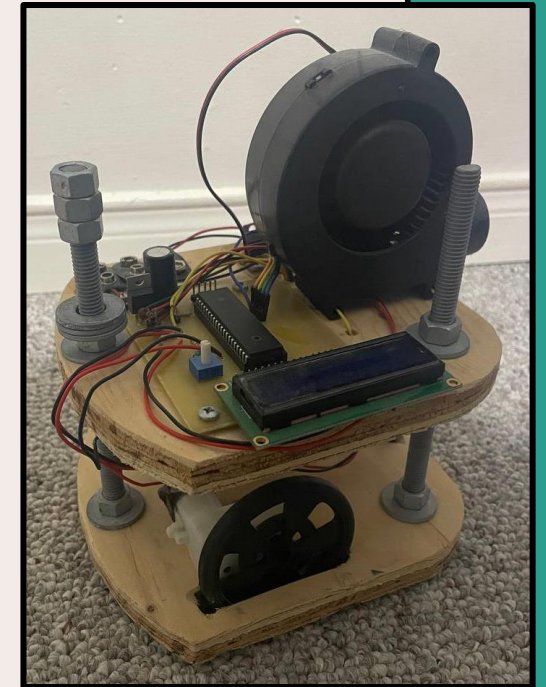
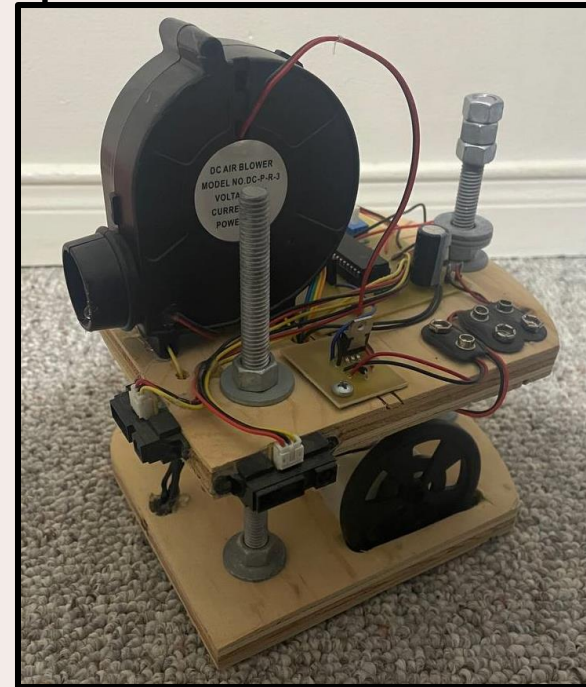
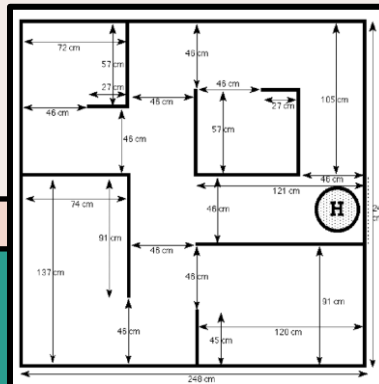
Terasic DE10-Lite development board used to create the project

Firefighter Robot

Code [here](#)

- Autonomous robot that searches for and extinguishes flame (candle)
- Utilized IR sensor for flame detection, 2 distance sensors and line follower along with a custom wall-tracking algorithm for fastest run
- Built upon the PIC16F887 and programmed in C compiled into BASIC
- Divided into 3 custom PCBs – Motor controller, Fan controller and Main processing unit

Sample maze the robot would have to clear. The lit candle can be in any of the rooms and the robot must locate and extinguish in fastest possible time.

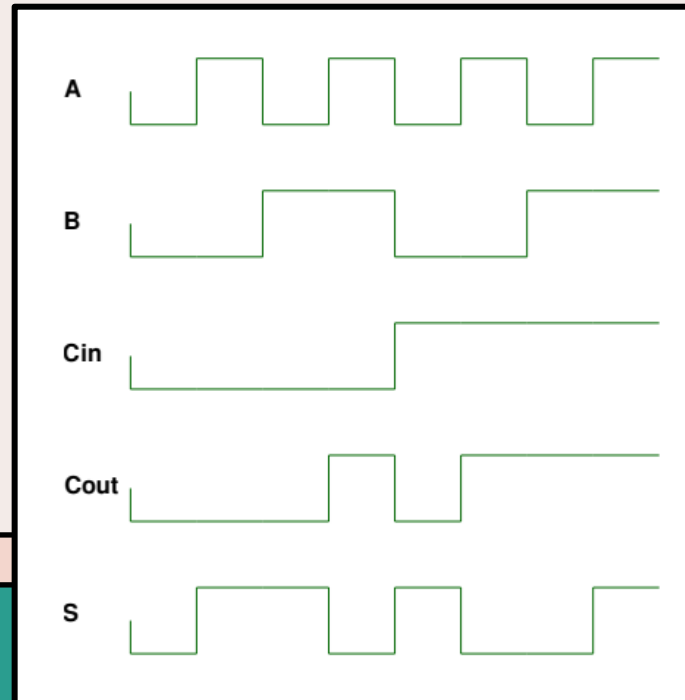


Fire Fighter Robot – on the 2nd layer you can see the fan facing forward with IR sensor at the nozzle connected to the fan controller pcb, behind the fan is the main processing unit with the PIC chip; on the 1st layer are 2 motorized wheels and at the front there is an LED and another IR sensor for line following

VHDL Compiler

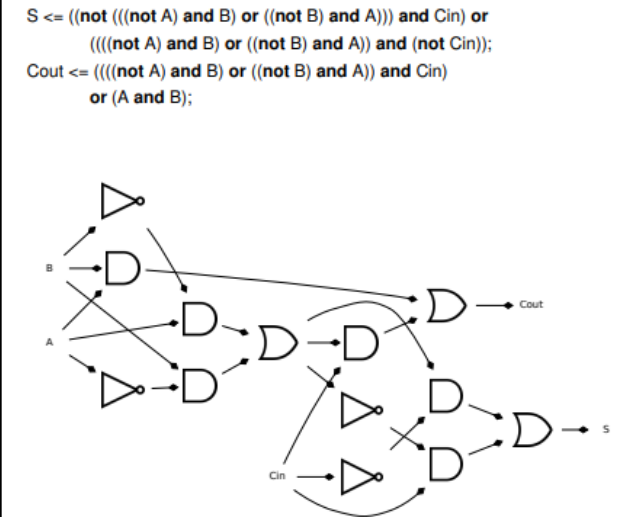
Code can be provided upon request

- A Java Compiler for creating combinational VHDL circuits
- Can efficiently optimize Boolean algebra logic using combinational synthesis
- Comes with a Logic Analyzer inspired visualizer for circuit output and another visualizer for circuit logic
- 8000 Line Project



```
entity full_adder is port (  
    A, B, Cin: in bit;  
    S, Cout: out bit  
);  
end full_adder;  
  
architecture full_adder_arch of full_adder is  
begin  
    S <= (A xor B) xor Cin;  
    Cout <= ((A xor B) and Cin) or (A and B);  
end full_adder_arch;
```

A: 0 1 0 1 0 1 0 1;
B: 0 0 1 1 0 0 1 1;
Cin: 0 0 0 0 1 1 1 1;



Full-Adder Circuit Example – On the left is the source code for the circuit (VHDL) along with the input waveform, on the right is the Boolean formulas generated from source code as well as the gates for full adder generated from formulas. On the bottom is the Input and output waveforms for full adder, generated from formulas.

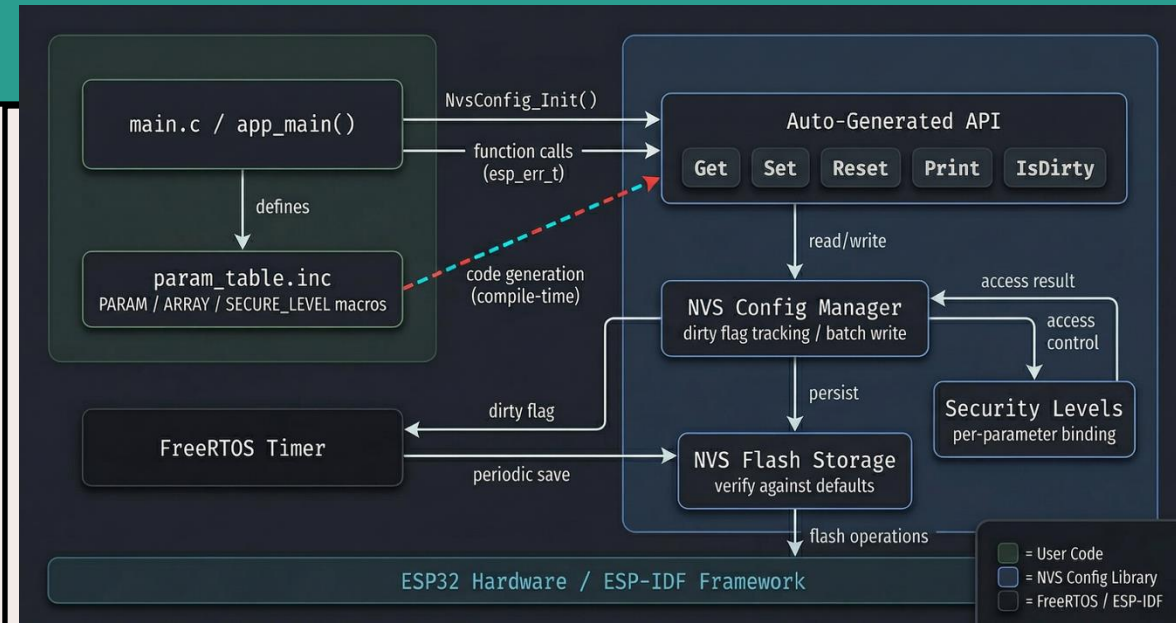
NVS Config

Published to ESP Registry

Code [here](#) | ESP Component [here](#)



- Published a production-grade ESP-IDF component with 7 buildable examples covering basic usage, arrays, interactive UART console, registry iteration, change callbacks, security levels, and stress testing
- Built a multi-pass C macro system that generates type-safe getters, setters, resetters, and print functions at compile time while also emitting a runtime parameter registry for generic, type-agnostic operations
- Added thread-safe FreeRTOS mutex protection, per-parameter and global change callbacks, role-based access control, schema versioning, and wear-level tracking to make the library robust in real firmware
- Validated the component with 163 host-based unit tests and documented 96% line coverage 75% branch coverage



hmolavi/nvs_config

1.0.0

uploaded 2 weeks ago

nvs-config makes creation and handling of NVS parameters effortless.

Readme

Dependencies 1

Dependents 0

README

latest version 1.0.0 Espressif Components GitHub hmolavi/nvs config

NVS Config Library

The **NVS Storage Config Library** is an ESP-IDF component designed to simplify the creation, management, and persistence of application parameters using Non-Volatile Storage (NVS). It supports both scalar values and array parameters, provides automatic verification against default values, and enables periodic saving of modified parameters using a FreeRTOS timer.



Toyota Challenge – Computer Vision Challenge Winner

Post with demo video [here](#) | Code [here](#)



- Won 1st Place (Top Prize / 300+ entrants) in Toyota CV Challenge for fastest completion time
- Developed real-time C++/Orbbec Astra system for 3D perception & precise vehicle tracking
- Implemented optimized C++ algorithms using depth data for low-latency object & feature detection
- Achieved high-precision event triggering based on real-time tracking; won Best Presentation award

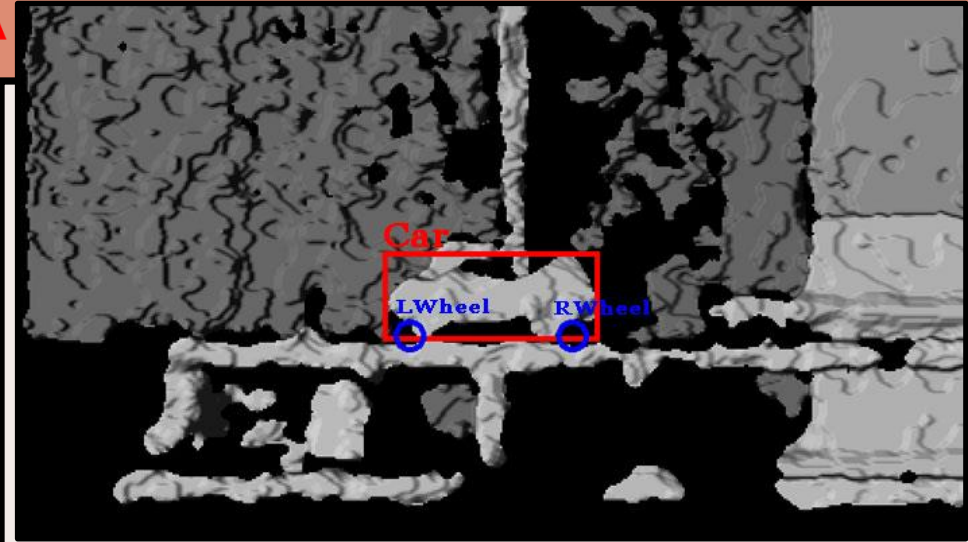
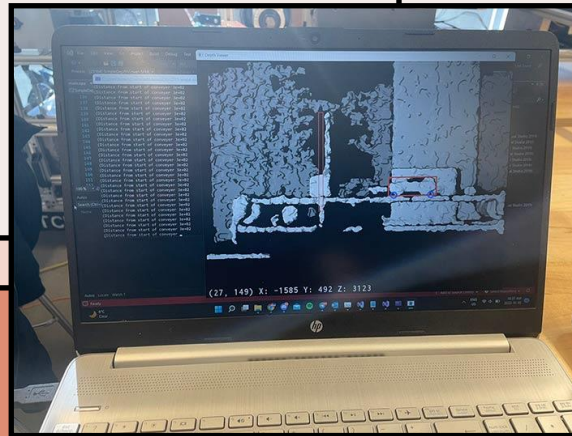


Photo taken from Live Demo: Car Detected !



Toyota Challenge – Fleet Control Sole Completer

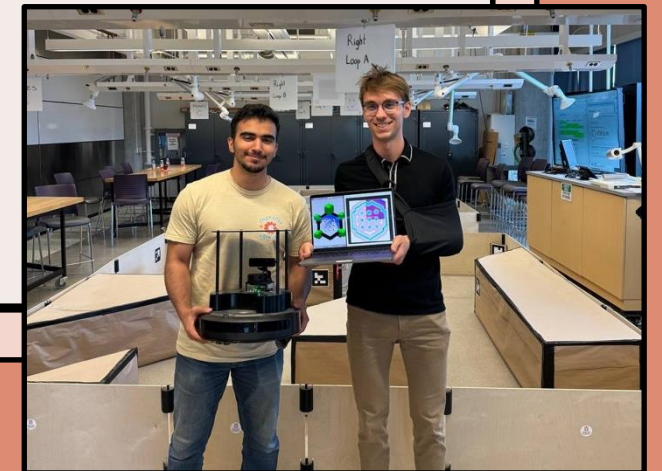
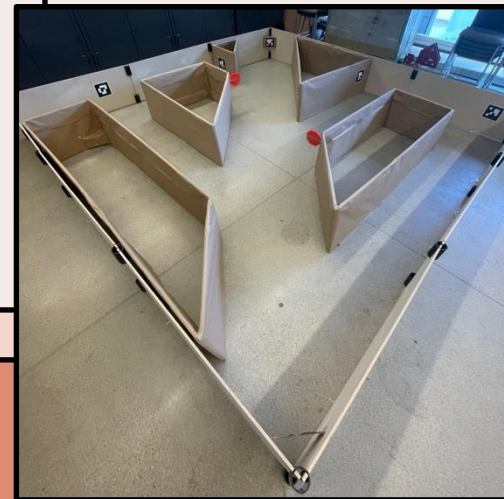
Demo video [here](#) | Code [here](#)



- Sole team completing all stages in Toyota Fleet Challenge; recognized for advanced AI (SLAM/CV) usage
- Engineered scalable multi-agent TurtleBot system in ROS2/Gazebo via extended Python control wrappers
- Implemented SLAM-based fleet coordination & ML vision for stop sign detection and response logic
- Integrated sensor fusion & complex decision logic within ROS2 for robust simulated robot navigation



TurtleBots (modified Roomba) used in the Challenge



Thankyou
for your considerations